

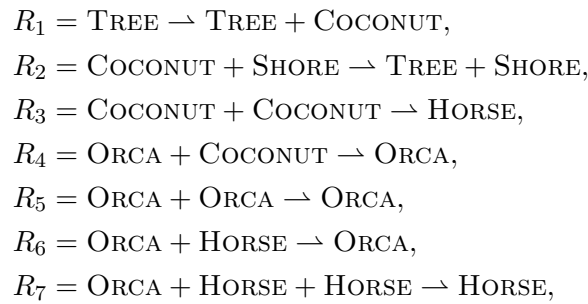
# Natural Computing SoSe25

## Exercise Sheet 6 (Solution) — July 10th, 2025

Thomas Gabor, Maximilian Zorn

### 1 Orcas and Coconuts

For the following tasks we define an artificial chemistry  $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$  where  $\mathcal{X} = \{\text{COCONUT}, \text{TREE}, \text{ORCA}, \text{HORSE}, \text{SHORE}\}$ ,  $\mathcal{R} = \{R_1, \dots, R_7\}$  with



and  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

(a) Give a possible evolution of the soup

$$X_0 = \{\text{COCONUT}, \text{SHORE}, \text{ORCA}\}$$

for as many time steps as necessary until a HORSE appears. (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.)

| time step | chosen rule | current soup                          |
|-----------|-------------|---------------------------------------|
| 0         | none        | {COCONUT, SHORE, ORCA}                |
| 1         | $R_2$       | {TREE, SHORE, ORCA}                   |
| 2         | $R_1$       | {TREE, COCONUT, SHORE, ORCA}          |
| 3         | $R_1$       | {TREE, COCONUT, COCONUT, SHORE, ORCA} |
| 4         | $R_3$       | {TREE, HORSE, SHORE, ORCA}            |
| 5         |             |                                       |

(b) Let  $X = \{\text{ORCA}, \text{ORCA}, \text{HORSE}, \text{HORSE}\}$ . **What is  $\mathcal{A}(X, \mathcal{R})$ ? What are all possible final soups if we keep running reactions on  $X$  for as long as we can?**

$$\mathcal{A}(X, \mathcal{R}) = \{(\{\text{ORCA}, \text{ORCA}\}, R_5), \\ (\{\text{ORCA}, \text{HORSE}\}, R_6), \\ (\{\text{ORCA}, \text{HORSE}, \text{HORSE}\}, R_7)\}$$

We will end up with the soup  $\{\text{ORCA}\}$  or  $\{\text{HORSE}\}$ .

(c) Now let  $X = X_{\text{Orcas}} \uplus \{x\}$  where  $X_{\text{Orcas}}$  is a multi-set consisting of 100 ORCA particles and  $x \in \mathcal{X}$  is one single mystery particle. We now run studies on the soup  $X$  for an extremely long time and every time we do so, we observe that in late soups there are no ORCA particles anymore. **What is the only plausible mystery particle  $x$ ? State your reasoning.**

$x = \text{TREE}$  because trees cannot die out and keep on producing coconuts, which keep on producing Horses, which eventually have a chance of killing Orcas.

## 2 DNA-Soup

In the lecture we will discuss an artificial chemistry  $(\mathcal{X}, \mathcal{R}, A)$  for a DNA-inspired soup as shown below. We assume that the bindings between DNA strands are labeled with elements from  $\{A, C, G, T\}$ .<sup>1</sup> In our case we have also defined  $-A = T$  and  $-C = G$  as A binds with T and C binds with G.

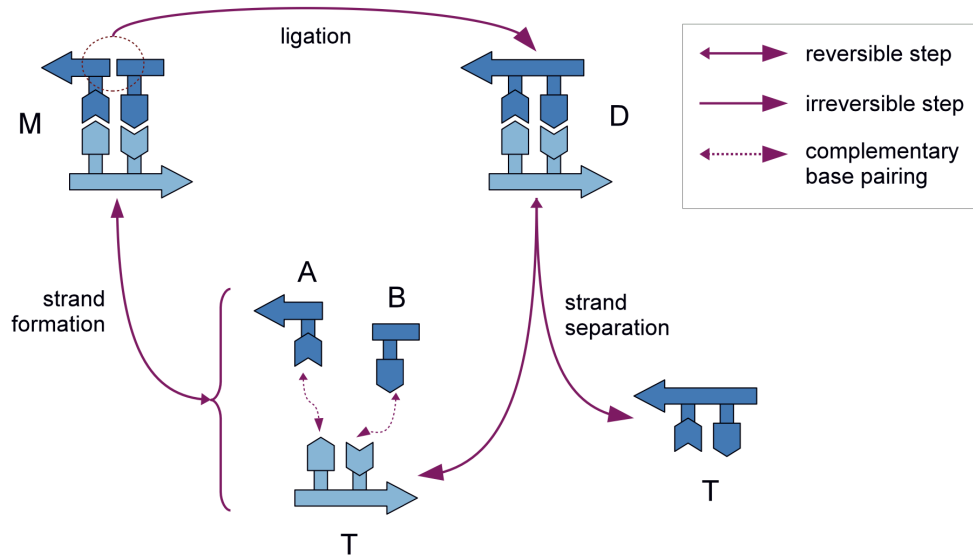


Figure 1: A model of the minimal self-replicator demonstrated by Von Kiedrowski (*A self-replicating hexadeoxynucleotide*. Angewandte Chemie International Edition in English 25.10, 1986). Taken from *Artificial Chemistries* (Banzhaf, Yamamoto. MIT Press, 2015), Ch. 6.2, Fig. 6.6

Let  $\mathcal{X} = \{A, C, G, T\}^{2 \times 2} \cup \{A, C, G, T\}^2 \cup \{A, C, G, T\}^1$ .

<sup>1</sup>These names are naturally inspired by the real-life DNA base pairs guanine–cytosine and adenine–thymine. Also see <https://en.wikipedia.org/wiki/DNA>.

Let  $\mathcal{R} = \{R_{1,-,-}, R_{2,-,-}, R_{3,-,-}, R_{4,-}\}$  with

$$\begin{aligned}
R_{1,X,Y} &= (X \ Y) + (-X) + (-Y) && \xrightarrow{0.5} && \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix} \\
R_{2,X,Y} &= \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix} && \xrightarrow{\quad} && (X \ Y) + (-Y \ -X) \\
R_{3,X,Y} &= (X) + (Y) && \xrightleftharpoons[0.01]{0.01} && (X \ Y) \\
R_{4,X} &= \emptyset && \xrightarrow{0.1} && (X)
\end{aligned}$$

for all  $X, Y \in \{A, C, G, T\}$ .

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R})$ .

(a) **Give an evolution of the soup**

$$X_0 = \{C, (T \ G)\}$$

**for all reaction steps indicated in the table.** (In our case we are always ‘lucky’ enough for our intended reaction to succeed; hence you may ignore the chance  $p$  in any reaction with  $\xrightarrow{p}$  for now.)

| time step | chosen rule               | current soup                                                    |
|-----------|---------------------------|-----------------------------------------------------------------|
| 0         | none                      | $\{C, (T \ G)\}$                                                |
| 1         | $R_{4,A}$                 | $\{C, A, (T \ G)\}$                                             |
| 2         | $R_{3,A,C}^{\rightarrow}$ | $\{(A \ C), (T \ G)\}$                                          |
| 3         | $R_{3,T,G}^{\leftarrow}$  | $\{(A \ C), T, G\}$                                             |
| 4         | $R_{1,A,C}$               | $\left\{ \begin{pmatrix} A & C \\ T & G \end{pmatrix} \right\}$ |
| 5         | $R_{2,A,C}$               | $\{(A \ C), (G \ T), \}$                                        |

(b) After working with the above soup encoding, a colleague proposed a slightly altered definition for the state space  $\mathcal{X}'$ . “Instead of using different types for single, smaller, or combined molecules, it might be more helpful to encode DNA molecules as fixed-size matrices that can contain empty fields.”, he said. Give an adjusted ruleset  $\mathcal{R}'$ , so that the reaction of task (a) still functionally works the same, but accomodates the state space

$$\mathcal{X}' = \{A, C, G, T, 0\}^{2 \times 2}.$$

Let  $\mathcal{X}' = \{A, C, G, T, 0\}^{2 \times 2}$ .

Let  $\mathcal{R}' = \{R_{1,-,-}, R_{2,-,-}, R_{3,-,-}, R_{4,-,-}\}$  with

$$R'_{1,X,Y} = \begin{pmatrix} X & Y \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} -X & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} -Y & 0 \\ 0 & 0 \end{pmatrix} \rightarrow^{0.5} \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix}$$

$$R'_{2,X,Y} = \begin{pmatrix} X & Y \\ -X & -Y \end{pmatrix} \rightarrow \begin{pmatrix} X & Y \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} -Y & -X \\ 0 & 0 \end{pmatrix}$$

$$R'_{3,X,Y} = \begin{pmatrix} X & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} Y & 0 \\ 0 & 0 \end{pmatrix} \rightleftharpoons_{0.01}^{0.01} \begin{pmatrix} X & Y \\ 0 & 0 \end{pmatrix}$$

$$R'_{4,X} = \emptyset \rightarrow^{0.1} \begin{pmatrix} X & 0 \\ 0 & 0 \end{pmatrix}$$

for all  $X, Y \in \{A, C, G, T\}$ .

Let  $A(X) \sim \mathcal{A}(X, \mathcal{R}')$ .

(c) Give  $X'_0$ , i.e., the counterpart to our initial soup

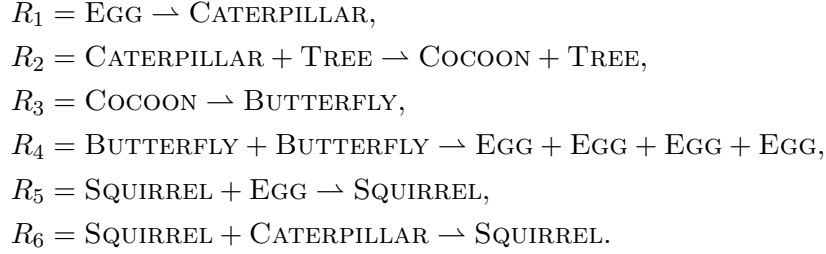
$$X_0 = \{C, (T \ G)\}$$

encoded according to our altered state-space definition  $\mathcal{X}'$ .

$$X'_0 = \left\{ \begin{pmatrix} C & 0 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} T & G \\ 0 & 0 \end{pmatrix} \right\}$$

### 3 Squirrels and Butterflies

For the following tasks we define an artificial chemistry  $\mathcal{B} = (\mathcal{X}, \mathcal{R}, A)$  where  $\mathcal{X} = \{\text{EGG}, \text{CATERPILLAR}, \text{COCOON}, \text{BUTTERFLY}, \text{TREE}, \text{SQUIRREL}\}$  and  $\mathcal{R} = \{R_1, \dots, R_6\}$  where



For the reactor function  $A$  we assume that it first picks a uniform random rule  $R \in \mathcal{R}$  and then checks if all particles necessary to apply  $R$  exist within the current soup  $X_t$  at time  $t$ . If such particles exist,  $A$  picks a random set of suitable particles  $X'$  to match the left hand side of rule  $R$  and then returns  $(X', R)$ . Whenever suitable particles for  $R$  do not exist within  $X_t$ ,  $A$  again picks a uniform random rule  $R \in \mathcal{R}$  and tries to use that.

(a) **Give a possible evolution of the soup**

$$X_0 = \{\text{CATERPILLAR}, \text{BUTTERFLY}, \text{TREE}\}$$

**for as many time steps as necessary until an EGG appears.** (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.)

| time step | chosen rule | current soup                   |
|-----------|-------------|--------------------------------|
| 0         | none        | {CATERPILLAR, BUTTERFLY, TREE} |
| 1         | $R_2$       | {TREE, BUTTERFLY, COCOON}      |
| 2         | $R_3$       | {TREE, BUTTERFLY, BUTTERFLY}   |
| 3         | $R_4$       | {TREE, EGG, EGG, EGG, EGG}     |
| 4         |             |                                |
| 5         |             |                                |

(b) **Give initial populations  $X_0^{stay}, X_0^{grow}, X_0^{shrink}$  so that when we evolve these populations separately, i.e., we compute  $X_t^{stay}, X_t^{grow}, X_t^{shrink}$ , respectively, for increasingly large values of  $t$  by applying the artificial chemistry  $\mathcal{B}$ , it holds that  $|X_t^{stay}|$  remains constant for all  $t$ ,  $|X_t^{grow}|$  eventually increases with sufficiently large  $t$ , and  $|X_t^{shrink}|$  eventually shrinks with sufficiently large  $t$ .**

$$\begin{aligned}X_0^{stay} &= \emptyset \\X_0^{grow} &= \{\text{BUTTERFLY}, \text{BUTTERFLY}\} \\X_0^{shrink} &= \{\text{SQUIRREL}, \text{EGG}\}\end{aligned}$$

(c) For this task, consider a soup of artificial chemistry  $\mathcal{B}$  with the initial population

$$X_0 = \{\text{EGG}, \text{SQUIRREL}, \text{TREE}\}.$$

**What are the chances that anywhere in the future evolution of  $X_0$  a BUTTERFLY appears? Show your reasoning.** You can assume that all executions of  $A$  eventually terminate. You can draw all possible paths for the evolution of  $X_0$  to derive the chances. Hint: Review exactly which behavior  $A$  uses.

```

{EGG, SQUIRREL, TREE}
| [R1], R5          1/2 chance
{CATERPILLAR, SQUIRREL, TREE}
| [R2], R6          1/2 chance
{COCOON, SQUIRREL, TREE}
| [R3]              1/1 chance
{BUTTERFLY, SQUIRREL, TREE}

```

Since this is the only path, the overall chances of a BUTTERFLY appearing are  $1/2 \cdot 1/2 \cdot 1 = 1/4$ .

(d) For this task, consider another soup of artificial chemistry  $\mathcal{B}$  with the initial population

$$X_0 = \{\text{EGG}, \text{SQUIRREL}, \text{SQUIRREL}, \text{TREE}\},$$

i.e., this time starting with two SQUIRRELS instead of only one. **Does this addition of another SQUIRREL affect the chances of a BUTTERFLY appearing, which you computed for task (c)? Give the chance and show your reasoning.**

Hint: Review exactly which behavior  $A$  uses.

No, the chances are not affected because  $A$ , as it is defined above, always chooses the *rule first*, which does not depend on the amount of particles available. Since we do not change the number of applicable rules, we also do not change the chance of a BUTTERFLY appearing.